

Robotaksi BT — Tam Mimari Analiz Raporu

0. ÖNEMLİ: Şu An 2 Ayrı Sistem Var

Projenin `behaviour_trees/` klasöründe **2 farklı nesil** ağaç var:

Nesil	Dosyalar	Durum
ESKİ (Waypoint)	<code>general_driving_bt.xml</code> , <code>round1_bt.xml</code> , <code>round2_bt_v2.xml</code> , <code>round2_bt_scenario2_final.xml</code> , <code>round3_bt_final.xml</code>	Eski sistem — kullanılmıyor
YENİ (Segment)	<code>segment_bt.xml</code>	Aktif sistem — geliştiriliyor

Soru: "Round 3 kurallarını takip ediyor mu?" → **Hayır, takip etmiyor.** Round 3 ağacı eski sistemde yazılmıştı. Yeni segment ağacına henüz taşınmadı. Bu raporun sonunda bunun nasıl çözüleceği anlatılıyor.

1. ESKİ SİSTEM — Nasıl Çalışıyordu?

Çalışma Mantığı (Waypoint tabanlı)

Görev geldi → Nav2 "noktaya git" →
Giderken reaktif olarak tehlikeleri kontrol et →
Tehlike bitince devam et

Tüm eski ağaçlar aynı yapıyı kullanıyor:

```
RecoveryNode (3 deneme hakkı):  
└─ ReactiveFallback (her tick, öncelik sırasına göre):  
    ├── P1: Dinamik engel var mı? → bekle  
    ├── P2: Statik engel var mı? → şerit değiştir veya bekle  
    ├── P3: Trafik ışığı var mı? → bekle  
    ├── P4: Trafik levhası var mı? → levhaya göre davran  
    └─ DEFAULT: NavigateToPose ← Nav2'ye "hedefe git" de
```

ReactiveFallback = her saniye baştan tarar. Yani robot sürüyorken bile P1, P2, P3, P4 sürekli sorgulanır. Tehlike çıkınca NavigateToPose iptal edilir, tehlike düzelince yeniden başlar.

Trafik Levhası Nasıl Ele Alınıyor? (Eski Sistem)

`P4_TrafficSigns` bloğu bir Fallback — içindeki her levha tipi sırayla kontrol edilir, ilk eşleşen çalışır:

```
P4_TrafficSigns (Fallback):  
└─ Şerit düzenleme levhası (TT-1/TT-2...) → LaneRegulationAction → yeni hedefe git  
└─ DUR tabelası → StopSignAction (tam dur, bekle,  
devam)  
└─ Otobüs durağı levhası → BusStopAction  
└─ Yaya geçidi levhası → SlowDownAction  
└─ Dönel kavşak levhası → RoundaboutAction  
└─ Tünel levhası → TunnelAction
```

└ Yanından geç levhası (TT-36/37)	→ PassByAction
└ Girilmez levhası (TT-26)	→ NoEntryAction (güzergah değiştir)
└ Yasak dönüş levhası (TT-4/TT-35)	→ ForbiddenTurnAction
└ Zorunlu dönüş levhası (TT-38/39)	→ MandatoryTurnAction

Önemli sorun: Bu sistemde levhalar **reaktif** işleniyor — robot levhayı görünce ne yapacağını biliyor ama nerede göreceğini, hangi kavşakta hangi levhanın olduğunu **önceden bilmiyor**. Bu yüzden bazen yanlış karar veriyor.

Round 1, 2, 3 Farkları

Dosya	Fark
general_driving_bt.xml	Temel şablon, sadece sürüş
round1_bt.xml	Round 1 versiyonu (minimal levha desteği)
round2_bt_v2.xml	Round 2 — ışık ve levha desteği eklendi
round2_bt_scenario2_final.xml	Round 2 senaryo 2 — park ve yolcu desteği eklendi
round3_bt_final.xml	Round 3 — tüm levhalar, tünel, dönel kavşak eklendi

Ama hepsi aynı temel yapı üzerinde. Sadece içine eklenen levha tipleri farklı.

2. YENİ SİSTEM — Nasıl Çalışıyor?

Temel Fark: "Nereye Gideceğini Önceden Biliyor"

Eski sistemde robot "levhayı görünce tepki ver" derdeydi. Yeni sistemde:

Harita = Graf (düğümler + segmentler)

Başlamadan önce:

LoadMission → tüm rotayı hesapla → segment listesi oluştur

Segment 1: LANE_FOLLOW (düz yol, 50m)

Segment 2: INTERSECTION (kavşak, sol dönüş)

Segment 3: LANE_FOLLOW (düz yol, 30m)

Segment 4: PASSENGER_STOP (yolcu durağı, Durak_1)

...

Robot artık "bu noktada kavşak var, soldan gelecek levhaya dikkat et" diye **önceden biliyor**.

Trafik Levhası Nasıl Ele Alınıyor? (Yeni Sistem)

Yeni sistemde levhalar **segment bazında** ele alınır, tüm yolda reaktif tarama değil:

Seg_Intersection ağacı çalışınca:

1. Hızı intersection_speed'e düşür (0.30 m/s)

2. Eğer Tur ≥ 2 ise:

a. HandleTrafficLight → ışığa bak

b. HandleStopSign → DUR tabelası var mı?

c. ValidateTurn → dönüş yönü levhayla çelişiyor mu?
3. SafeDrive ile kavşağı geç

Yani levhalar **sadece kavşakta** (INTERSECTION segmentinde) kontrol ediliyor — bu daha doğru ve daha güvenli.

3. TÜM AĞAÇLAR — Detaylı Çalışma Mantığı

A. MainTree (Giriş noktası)

```
BAŞLA
|
|— 5x hız parametresi ayarla (SetBlackboard)
|   cruise=0.60, intersection=0.30, roundabout=0.30, tunnel=0.30, parking=0.20
|
|— LoadMission
|   GeoJSON dosyasını oku
|   + segment_map.yaml'dan grafi yükle
|   → rota oluştur (segment dizisi)
|   → route, route_size, seg_index=0 blackboard'a yaz
|
|— DriveWithRecovery (Fallback)
|   |— DriveAllSegments ← rotayı yürüt
|   |— Recovery          ← başarısız olursa kurtarma (BackUp+Spin)
|
|— FinishMission → StopVehicle
```

Kendi kendine gidiyor mu? → Gitmesi için LoadMission ve FollowLaneSegment implement edilmeli. Şu an stub (sahte). Ama mimari hazır.

B. DriveAllSegments (Döngü motoru)

```
TEKRAR (sonsuz):
|— Daha segment var mı? (HasMoreSegments)
|   Hayır → döngüden çık, SUCCESS dön
|   Evet  → devam et
|
|— Şu anki segmenti al (GetCurrentSegment)
|   route[seg_index] → seg_type, seg_goal, seg_meta
|
|— Segmenti çalıştır (ExecuteSegment)
|   seg_type'a göre doğru alt-ağacı seç
|
|— Sonraki segmente geç (AdvanceSegment)
|   seg_index++
```

Döngü bitince (tüm segmentler tamamlandı):

- HasMoreSegments FAILURE → Fallback'teki Inverter(HasMoreSegments) SUCCESS → DriveAllSegments başarılı biter

C. ExecuteSegment (Switch/Case router)

Fallback (ilk eşleşen çalışır):

```
|
├─ seg_type == "LANE_FOLLOW"?    → Seg_LaneFollow
├─ seg_type == "INTERSECTION"?   → Seg_Intersection
├─ seg_type == "ROUNDABOUT"?    → Seg_Roundabout
├─ seg_type == "TUNNEL"?         → Seg_Tunnel
├─ seg_type == "PASSENGER_STOP"? → Seg_PassengerStop
├─ seg_type == "LANE_CHANGE"?    → Seg_LaneChange
└─ seg_type == "PARKING"?        → Seg_Parking
```

Her `IsSegmentType` node'u blackboard'daki `seg_type` değişkenini kontrol eder. Eşleşmeyenler FAILURE döner, Fallback bir sonrakine geçer.

D. SafeDrive (Güvenlik + Hareket primitifi)

ReactiveSequence (her tick ikisini de çalıştır):

```
|
├─ SafetyReflexes ← önce güvenlik
└─ FollowLaneSegment ← sonra hareket
```

ReactiveSequence kritik: Normal Sequence'tan farkı şu:

- Normal Sequence: A bitti → B çalış → C çalış → bitti
- ReactiveSequence: Her tick A'ya bak → A SUCCESS ise B'ye bak → B RUNNING ise dur

Yani robot sürüyor (FollowLaneSegment RUNNING), aynı anda SafetyReflexes her tick kontrol ediliyor. Tehlike çıkınca FollowLaneSegment durduruluyor.

E. SafetyReflexes (4 katmanlı güvenlik)

ReactiveSequence:

```
|
├─ Fallback [Katman 1 – Acil Stop]:
|   └─ NOT EmergencyStopRequested ← buton basılı değil → geç
|       └─ KeepRunningUntilFailure ← basılıysa: StopVehicle sonsuz çalışır
|           └─ StopVehicle (KeepRunningUntilFailure SUCCESS dönmez)
|
├─ Fallback [Katman 2 – Yaya]:
|   └─ NOT PedestrianAhead ← yaya yok → geç
|       └─ Sequence:
|           └─ StopVehicle ← dur
|               └─ WaitForClear ← yaya geçene kadar bekle
|
├─ Fallback [Katman 3 – Dinamik Engel]:
|   └─ NOT DynamicObstacleAhead ← engel yok → geç
|       └─ Sequence:
|           └─ StopVehicle
|               └─ WaitForClear(dynamic)
|
└─ Fallback [Katman 4 – Statik Engel]:
```

```
└─ NOT StaticObstacleInLane    ← engel yok → geç
└─ HandleStaticObstacle        ← şerit değiştir veya bekle
```

Öncelik mantığı: ReactiveSequence ilk FAILURE'da durur. Katman 1 RUNNING dönerse (acil stop basılı), 2,3,4 hiç çalışmaz. Yani acil stop her şeyin önüne geçer.

F. HandleStaticObstacle

```
Fallback:
└─ Sequence [Şerit değiştir]:
  │   └─ AvoidanceSpaceAvailable    ← yan şerit boş mu?
  │   └─ CalculateLaneChange        ← geçiş hedefini hesapla
  │   └─ FollowLaneSegment          ← hesaplanan hedefe git
└─ Sequence [Dur ve bekle]:
  │   └─ StopVehicle                ← yan şerit doluysa dur
  │   └─ WaitForClear(static)        ← engel çekilene kadar bekle
```

G. HandleTrafficLight

```
Fallback:
└─ NOT TrafficLightAhead          ← ışık göremiyorum → geç (başarılı)
└─ Sequence [Işık işleme]:
  │   └─ Fallback [Kırmızı mı?]:
  │     │   └─ NOT IsLightRed        ← yeşil → geç
  │     │   └─ Sequence [Kırmızıda dur]:
  │     │     │   └─ StopAtStopLine (5m tolerans)
  │     │     │   └─ WaitForGreenLight
  │     └─ ProceedOnGreen (max 5sn)
```

Puan mantığı (şartname): Kırmızıda geçersen -puan. Yeşilde 5sn içinde kalkmazsan -puan. Bu ağaç ikisini de önler.

H. HandleStopSign

```
Fallback:
└─ NOT StopSignAhead              ← tabela yok → geç
└─ StopAndProceed                 ← tam dur (2 saniye) sonra devam
```

I. ValidateTurn

```
Fallback:
└─ NOT TurnConflictsWithSigns    ← "yasak dönüş" veya "girilmez" yok → geç
└─ ForceSuccess:                  ← çelişki varsa rotayı yeniden planla
  │   └─ ReplanRoute(sign_conflict)
  │       route, route_size, seg_index güncellenecek
```

ForceSuccess neden var? ReplanRoute başarısız olsa bile (rota bulunamazsa) ağaç çökmemeli. ForceSuccess her durumda SUCCESS döndürür.

J. Seg_LaneFollow

Sequence:

```
|— SetMaxSpeed(cruise_speed=0.60)
|— RecordMissionPoint(seg_meta, 1.0m) ← görev noktası geçişini kaydet
|— SafeDrive ← güvenli şerit takibi
```

En basit segment. Hedefe güvenli şekilde git, geçişi kaydet (+30 puan).

K. Seg_Intersection

Sequence:

```
|— SetMaxSpeed(intersection_speed=0.30)
|
|— Fallback [Tur kapısı]:
|   |— NOT IsTrafficControlActive(tour) ← Tur 1: trafik kontrolü pasif → atla
|   |— Sequence [Tur 2-3: kontrol et]:
|       |— HandleTrafficLight
|       |— HandleStopSign
|       |— ValidateTurn
|
|— SafeDrive ← kavşağı geç
|— SetMaxSpeed(cruise_speed=0.60)
```

Tur mantığı: `IsTrafficControlActive` node'u `tour` değişkenine bakar. Tur 1 ise FALSE döner → Fallback'in ilk çocuğu (`NOT IsTrafficControlActive`) SUCCESS → trafik kontrol bloğu atlanır. Tur 2-3 ise TRUE döner → kontrol bloğu çalışır.

L. Seg_Roundabout

Sequence:

```
|— SetMaxSpeed(roundabout_speed=0.30)
|— YieldAtRoundabout ← dönel kavşakta araç akışına bak, boşluk bekle
|— SafeDrive ← dönel kavşakta güvenli sür (refleksler aktif!)
|— SetMaxSpeed(cruise_speed=0.60)
```

SafeDrive'in burada olması kritik — dönel kavşak içinde de yaya çıkabilir, dinamik engel olabilir.

M. Seg_Tunnel

Sequence:

```
|— SetMaxSpeed(tunnel_speed=0.30)
|— TurnHeadlights(on) ← far aç
|— SafeDrive ← tünelden geç
|— TurnHeadlights(off) ← far kapat
|— SetMaxSpeed(cruise_speed=0.60)
```

Şartname: Tünel içinde far açık olmalı (+200 puan). Tünel çıkışında kapatılmalı.

N. Seg_PassengerStop

```
Sequence:
├─ SafeDrive          ← durağa yaklaş
├─ StopVehicle        ← dur
├─
├─ Fallback [EnsureStopAccuracy – pozisyon düzeltme]:
│   ├─ CheckStopAccuracy(1.0m) ← 1m içinde mi?
│   └─ Sequence [Değilse düzelt]:
│       ├─ FollowLaneSegment ← düzeltme hareketi
│       ├─ StopVehicle
│       └─ CheckStopAccuracy ← tekrar kontrol
├─
├─ Dwell(min=15sn, max=20sn) ← yolcu bekleme süresi
└─ SignalPassengerEvent(seg_meta) ← yolcu al/bırak sinyali
```

Şartname: 1m hassasiyetle dur (+70/nokta). 15-20sn bekle.

O. Seg_Parking

```
Sequence:
├─ SetMaxSpeed(parking_speed=0.20)
├─ SafeDrive          ← park girişine sür
├─ RecordParkEntryReached ← giriş geçildi (+20 puan)
├─ FindParkingSlot → slot_pose ← uygun slot bul (P-3a var, P-1 yok)
└─ ExecuteParking(slot, 180sn) ← dik park manevrası (+80 puan)
```

P. Recovery

```
Sequence:
├─ IsStuck          ← robot sıkışık mı? (hız=0, hareket yok)
├─ BackUp(0.30m) ← 30cm geri git
└─ Spin(1.57rad) ← 90 derece dön
```

Recovery sadece DriveAllSegments başarısız olursa çalışır.

4. "Kendi Kendine Haritada Gidiyor mu?"

Şu an: HAYIR. Gitmesi için şu node'lar implement edilmeli:

1. **LoadMission** → GeoJSON'u okuyup gerçek rota oluşturmalı
2. **FollowLaneSegment** → Vision lane-follow veya Nav2 FollowPath çağırmalı
3. **HasMoreSegments** , **GetCurrentSegment** , **AdvanceSegment** → Bunlar basit, zaten çalışıyor sayılır
4. **StopVehicle** → `cmd_vel` sıfırlamalı

Teorik olarak: Yukarıdakiler implement edilirse evet, haritadaki segmentleri kendi kendine gezer.

5. Round 3 Kuralları Yeni Sisteme Taşındı mı?

HAYIR. Karşılaştırma:

Kural	Round 3 Eski Sistem	Yeni Segment Sistemi
Dinamik engel → dur/bekle	✓ DynamicObstacleCondition	✓ DynamicObstacleAhead (stub)
Statik engel → şerit değiştir	✓ CalculateLaneChangeAction	✓ HandleStaticObstacle (stub)
Trafik ışığı	✓ TrafficLightCondition	✓ HandleTrafficLight (stub)
DUR tabelası	✓ StopSignAction	✓ HandleStopSign (stub)
Dönel kavşak	✓ RoundaboutAction	✓ Seg_Roundabout (stub)
Tünel + far	✓ TunnelAction	✓ Seg_Tunnel (stub)
Girilmez/Yasak dönüş	✓ NoEntryAction, ForbiddenTurnAction	✓ ValidateTurn (stub)
Yolcu durağı (15-20sn)	⚠ BusStopAction (basit)	✓ Seg_PassengerStop (daha iyi!)
Pozisyon hassasiyeti (1m)	✗ Yok	✓ CheckStopAccuracy (stub)
Dönel kavşak içi güvenlik	✗ Yok	✓ SafeDrive içinde

Kural bazında yeni sistem Round 3'ten DAHA İYİ tasarlandı — sadece implementasyonlar stub.

6. Özet: Ne Çalışıyor, Ne Eksik

✓ Çalışıyor (mimari hazır):

- Tüm ağaç yapısı Groot'ta görülebiliyor
- Tur 1/2/3 ayrımı (IsTrafficControlActive)
- Güvenlik katmanı hiyerarşisi (acil > yaya > dinamik > statik)
- Levha işleme mantığı (HandleTrafficLight, HandleStopSign, ValidateTurn)
- Pozisyon düzeltme (EnsureStopAccuracy)
- Dönel kavşak içi güvenlik
- Build 0 hatayla geçiyor

✗ Stub (sahte, implement edilmeli):

- FollowLaneSegment → aracı gerçekten sürmüyor
- LoadMission → rota oluşturmuyor
- PedestrianAhead → kameraya bağlı değil
- DynamicObstacleAhead → lidar'a bağlı değil
- TrafficLightAhead → ışık algılamıyor
- StopVehicle → cmd_vel 'e bağlı değil
- TurnHeadlights → GPIO'ya bağlı değil
- Dwell → süreyi sayıyor ama yolcu olayı yok

- FindParkingSlot / ExecuteParking → park manevrası yok